

Hash 与直接定位

用槽位和冲突管理换取接近直接访问

408 · 数据结构 Topic DS-10

母问题：映射不是定位，冲突怎样成为状态的一部分？

散列的生成链是

Key → Hash Mapping → Collision → Resolution → Load State → Expected Cost.

哈希函数只给出候选槽位，不保证唯一；开放定址沿探测序列寻找空位，链地址把同槽 key 放入桶链。查找失败必须按同一探测规则走到“从未使用”或完整探测，删除不能简单清空开放定址槽位，否则会截断后续 key 的探测链。

本册定位与 Owner 边界

- **Owns**：散列函数、装填因子、开放定址/链地址、删除标记、重散列与成功/失败查找语义。
- **Uses**：Atlas Foundation 与 DS-B02 workload 比较；精确等值查询优先，不承诺范围顺序。
- **Stops**：有序索引归 DS09；具体工程哈希表的并发、缓存和安全策略只作 Extension。

目录

1	状态与冲突策略	2
2	成本与边界	2
3	代码与测试证据	2
4	做题控制协议	3

1 状态与冲突策略

开放定址槽位至少有 Empty、Occupied、Deleted 三态；链地址槽位可用容器表示。装填因子 $\alpha = n/m$ 越高，冲突和探测长度通常越大；超过阈值应扩容并重新插入，不能直接把旧下标复制到新表。删除标记仍占探测路径，重散列时可被清除。

为什么删除后不能变 Empty

若 $h(a) = h(b)$ ，先插入 a 再插入 b ， b 可能探测到下一个槽。删除 a 后若把槽清空，查找 b 看到第一个 Empty 就会错误失败；Deleted 表示“继续探测，但此处可复用”。

2 成本与边界

均匀散列、合理装填因子和独立探测假设下，开放定址成功/失败查找期望接近常数；最坏可退化到 $O(m)$ 。哈希不保序，因此范围查询、前驱后继和有序遍历不能由平均 $O(1)$ 推出。

3 代码与测试证据

完整实现位于 code/ds10_hash_table.hpp，测试位于 code/ds10_hash_table_test.cpp。

Listing 1: 线性探测、删除标记与重散列

```

12 public:
13     explicit LinearHashTable(std::size_t capacity = 8) : slots_(capacity ?
        capacity : 1) {}
14     bool insert(int key) { if ((size_ + 1) * 10 >= slots_.size() * 7)
        rehash(slots_.size() * 2); return place(key); }
15     bool contains(int key) const { return locate(key).has_value(); }
16     bool erase(int key) { auto index = locate(key); if (!index) return false
        ; slots_[*index] = Slot::deleted(); --size_; return true; }
17     std::size_t size() const { return size_; }
18
19 private:
20     struct Slot { std::optional<int> value; bool tombstone = false; static
        Slot deleted() { return Slot{std::nullopt, true}; }; };
21     std::vector<Slot> slots_; std::size_t size_ = 0;
22     std::size_t hash(int key) const { return static_cast<std::size_t>(key >=
        0 ? key : -static_cast<long long>(key)) % slots_.size(); }
23     bool place(int key) { std::size_t first_deleted = slots_.size(); for (
        std::size_t step = 0; step < slots_.size(); ++step) { auto i = (hash(
        key) + step) % slots_.size(); if (slots_[i].value && *slots_[i].value
        == key) return false; if (!slots_[i].value) { if (first_deleted !=
        slots_.size()) i = first_deleted; slots_[i].value = key; slots_[i].
        tombstone = false; ++size_; return true; } if (slots_[i].tombstone &&
        first_deleted == slots_.size()) first_deleted = i; } return false; }
24     std::optional<std::size_t> locate(int key) const { for (std::size_t
        step = 0; step < slots_.size(); ++step) { auto i = (hash(key) + step)
        % slots_.size(); if (!slots_[i].value && !slots_[i].tombstone) return

```

```

    std::nullopt; if (slots_[i].value && *slots_[i].value == key) return i
    ; } return std::nullopt; }
25 void rehash(std::size_t capacity) { auto old = std::move(slots_);
    slots_.assign(capacity, Slot{}); size_ = 0; for (const auto& slot :
    old) if (slot.value) place(*slot.value); }
26 };
27 } // namespace ipara::ds10
28
29 #endif

```

Listing 2: 冲突、删除后查找与满表边界测试

```

8  assert(table.contains(5)); assert(table.erase(1)); assert(!table.
    contains(1));
9  assert(table.contains(5));
10 for (int value = 10; value < 20; ++value) assert(table.insert(value));
11 assert(table.size() == 11);
12 }

```

```

clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/
    ds10_hash_table_test.cpp -o /tmp/ds10_test
/tmp/ds10_test

```

4 做题控制协议

先问查询是否精确等值，再写 $h(k)$ 、冲突处理、装填因子和失败终止条件。开放定址题画探测序列，删除保留 tombstone；扩容题重新计算每个 key 的槽位。若题目要求范围或排序，转交 DS09/DS11。

压缩复原

五问：key 映射到哪里？冲突后走哪条路径？槽位有几种状态？失败何时停止？装填变大如何修复？